

LING12364: Mining Patterns in Data

Project 2: Frequent Subtrees Mining

1 Context

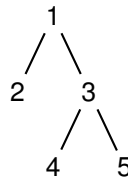
There exist several algorithms for mining frequent sequences as well as for mining frequent trees. Although these structures differ in complexity, they share common principles in terms of pattern growth and search space exploration.

In this project, you will adapt a **sequence mining algorithm** of your choice (such as PrefixSpan or GSP) to mine frequent **induced rooted subtrees** from a database of rooted ordered trees. An induced subtree preserves the parent-child relationships of the original tree.

2 Tree Representation

To represent a rooted ordered tree in a linear format, we use a string representation based on a pre-order traversal with a special value, -1 , to denote backtracking to the parent node.

Consider the following example tree:



The representation of this tree is: 1 2 -1 3 4 -1 5.

- We start at the root **1**.
- We visit the first child **2**.
- Since **2** has no children, we backtrack to **1** using **-1**.
- We visit the next child of **1**, which is **3**.
- We visit the first child of **3**, which is **4**.
- Since **4** has no children, we backtrack to **3** using **-1**.
- We visit the next child of **3**, which is **5**.

Note that trailing -1 s are often omitted in this representation.

The project will be done in Python and a template is provided in `sim_template.py`. You are free to modify this template as long as you respect the directives

2.1 The Tree Class

The `Tree` class represents a single rooted ordered tree. It uses two main lists to describe the structure:

- `labels`: A list of integers representing the label of each node.
- `parents`: A list of integers where each element at index i is the index of the parent of node i . The root of the tree has -1 as its parent.

For example, the tree 1 2 -1 3 4 -1 5 would be represented as:

```
labels = [1, 2, 3, 4, 5]
parents = [-1, 0, 0, 2, 2]
```

Upon initialization, the `Tree` class also computes:

- `self.children`: A list of lists where `children[i]` contains the indices of the children of node i .
- `self.label_index`: A dictionary mapping each label to the list of node indices that carry that label, allowing for fast lookup of starting points for subtrees.

2.2 The Dataset Class

The `Dataset` class is responsible for loading the database of trees from a file. It parses each line (a -1 delimited sequence) and converts it into a `Tree` object.

The parsing logic uses a stack-based approach:

- When a label is encountered, it is added to the `labels` list. Its parent is the node currently at the top of the stack. The new node index is then pushed onto the stack.
- When a -1 is encountered, it indicates that the current branch is finished, so the top node is popped from the stack.

3 Directives

The project must be completed in groups of two students and will be graded using INGINious.

You must implement:

1. The `is_valid_subtree` method in the `Tree` class.
2. A mining algorithm adapted from a sequence mining approach.

3.1 The Tree Checker

You are required to implement a `is_valid_subtree` method in the `Tree` class. This method will be tested independently on INGINious.

```
class Tree:
    def __init__(self, labels, parents):
        self.labels = labels
        self.parents = parents

    def is_valid_subtree(self, pattern_tree):
        """
        pattern_tree: list of labels in -1 delimited format
        returns: True if the pattern_tree is an induced rooted subtree of this tree
        """
        # TODO: Implement this method
        pass
```

3.2 The mining algorithm

You must implement a mine method in `miner.py`:

```
def mine(dataset_path, min_frequency):  
    """  
    dataset_path: path to the file containing trees (one per line)  
    min_frequency: float between 0 and 1  
    """  
    pass
```

4 Output

Your program must print every frequent subtree to the standard output in the following format:

```
[node1, node2, ..., -1, nodeK] (freq)
```

where `freq` is the relative frequency (support). The nodes should follow the `-1` delimited pre-order traversal. Note that trailing `-1`s are usually omitted.

5 Grading

You will be graded according to the following criteria:

- **Correctness and performance of your implementations (8/20).** This includes:
 - The correctness of the `is_valid_subtree` method (20% of implementation grade).
 - The correctness and performance of your mining algorithm (80% of implementation grade).
- **Quality and relevance of your report (12/20).** This includes the justification of your implementation choices, the analysis of your experimental results, and the overall clarity of your presentation.

6 Report

Your report must not exceed 4 pages and should be written in correct English. It must include:

- A description of how you adapted the sequence mining algorithm to trees.
- The key implementation choices you made (e.g., data structures used for the search).
- Any optimizations you implemented to improve performance.
- An experimental evaluation on the provided datasets, discussing execution time and scalability with respect to the support threshold.
- A clear and structured analysis of the results.

Be precise and concise. Use tables or graphics to depict your results where appropriate. Don't forget to include your group number and the names/NOMAs of all members.

7 Datasets

Two datasets are available in the `datasets/` directory. For each dataset, some example solutions for specific thresholds are provided in the `solutions/` directory (e.g., `small_1_0.1` for the `small_1` dataset at 10% frequency).

In particular, a **toy dataset** (`toy.trees`) and its corresponding solution are provided to help you with the initial debugging of your algorithm. We strongly encourage you to test your implementation locally using these examples before submitting to INGINIOUS.

8 Important Tips

- **Efficiency:** Tree mining can be computationally expensive. Think about how to prune the search space effectively.
- **Backtracking:** Properly handling the -1 symbol is key to correctly traversing and comparing trees.
- **Testing:** Use the provided `test_validdtree.py` script to verify your `Tree` class implementation.
- **INGInious:** Remember that INGInious is a grading tool, not a debugger. Ensure your code works locally first.